

DETECCIÓN DE CARRILES: CLÁSICO VS DEEP LEARNING

Caso Práctico 1 · Dossier del alumno

Máster en IA — AIC

Módulo: IA Aplicada al Vehículo Autónomo · 2026

Alcance y requisitos previos

Este es el primer caso práctico del módulo. Está diseñado para poder realizarse **sin conocimientos previos** en el uso de herramientas de programación como la terminal, los sistemas de control de versiones (GitHub) o los entornos de desarrollo. Cada concepto y cada instrucción se explican de forma detallada.

- La **primera parte** (preparación del entorno de trabajo) es la más extensa. Se realiza **una única vez**; los casos prácticos posteriores serán considerablemente más breves, dado que el equipo ya estará configurado.
- Se recomienda copiar los comandos **literalmente**, de uno en uno. No es necesario comprenderlos para que funcionen correctamente.
- Si el resultado obtenido no coincide con el descrito, se aconseja **no continuar** y consultar el apartado "Resolución de incidencias" o el foro del máster.

 **Duración estimada:** preparación del entorno ~45 min (solo la primera vez). Ejecución de la práctica ~1 hora.

Objeto de la práctica

Se pondrán en funcionamiento **dos detectores de carriles** sobre un mismo conjunto de imágenes de carretera, con el fin de **compararlos**:

1. Un método **clásico**, propio de la visión por computador de los años 90, basado en operaciones matemáticas sobre la imagen [sin inteligencia artificial moderna].
2. Un método basado en **inteligencia artificial**: una red neuronal previamente entrenada por el equipo docente.

Y se dará respuesta a una cuestión central:

¿Qué método ofrece mejores resultados y en qué situaciones resulta más adecuado cada uno?

No es necesario programar los detectores: **ambos se proporcionan ya implementados**. La labor del alumno consiste en ejecutarlos, observar su comportamiento y extraer conclusiones.

Objetivos de aprendizaje

Al finalizar la práctica, el alumno será capaz de:

1. Ejecutar un procedimiento de visión clásica **paso a paso** y comprender la función de cada operación.
2. Cargar y aplicar un modelo de inteligencia artificial **ya entrenado**, respetando sus requisitos de entrada y salida.
3. Medir el **tiempo de cómputo real** de cada método y valorarlo frente a las exigencias de tiempo real (30 imágenes por segundo → 33 milisegundos por imagen).
4. Comparar ambos enfoques mediante **evidencia cuantitativa**, no impresiones subjetivas.
5. Identificar las **limitaciones** de cada método y proponer mejoras concretas.

Estructura del trabajo

El desarrollo se organiza en varios archivos denominados **notebooks** (se explican más adelante), que deberán abrirse en el orden indicado:

Archivo	Función	Duración
01_setup	Verifica que la instalación es correcta	5 min
02_clasico_canny_hough	Detector clásico	15 min
03_deep_learning	Detector con inteligencia artificial	10 min
04_comparativa	Comparación y gráficas	10 min
05_preguntas	Cuestionario de entrega	10 min

Como resultado se entregará **un documento PDF** con los resultados y las respuestas. El contenido exacto de la entrega se detalla al final de este dossier.

PARTE 0

Preparación del entorno de trabajo (solo la primera vez)

Visión general del procedimiento

Se trata de una serie de pasos mecánicos, equivalentes a la instalación de cualquier programa. Se ejecutarán en el siguiente orden:

1. **Descarga del material** de la práctica desde GitHub.
2. **Apertura de la terminal** (la ventana para introducir instrucciones).
3. **Instalación de Python** (el lenguaje en el que están escritos los programas).
4. **Creación de una cuenta de GitHub** (necesaria para la herramienta de descarga).
5. **Instalación y conexión de `gh`** (la herramienta que descarga imágenes y modelo).
6. **Acceso a la carpeta** de la práctica.
7. **Creación de un entorno e instalación de las dependencias** necesarias.
8. **Descarga de las imágenes y del modelo de inteligencia artificial.**
9. **Apertura de Jupyter** y ejecución del primer notebook.

Se recomienda marcar  cada paso a medida que se completa.

Tres términos de uso frecuente

Terminal (o "consola"): una ventana en la que, en lugar de utilizar el ratón, se **introducen instrucciones escritas** y se pulsa Intro. Es, simplemente, texto.

Python: el "lenguaje" en el que están escritos estos programas. Debe instalarse, del mismo modo que se instala un procesador de textos para poder abrir un documento `.docx`.

GitHub: una plataforma web para almacenar y compartir archivos de proyectos. En ella se encuentran las imágenes y el modelo de inteligencia artificial de esta práctica.

Paso 1 · Descarga del material desde GitHub

Todo el material de la práctica está en un repositorio **público** de GitHub. Cualquiera puede descargarlo; no se requiere permiso.

1. Acceda a <https://github.com/codx-digital/master-ia-practicas>
2. Pulse el botón verde **Code** y, en el menú desplegable, **Download ZIP**.
3. Abra el archivo descargado para **descomprimirlo** y sitúe la carpeta resultante en un lugar accesible (por ejemplo, el **Escritorio**).

Dentro encontrará la carpeta **CP1-carriles**, con los notebooks, el dossier y todo lo necesario para el resto de los pasos.

Paso 2 · Apertura de la terminal

Es la ventana en la que se introducirán las instrucciones. Ábrala según su sistema operativo:

En Windows:

- Pulse la tecla **Windows**, escriba **PowerShell** y ábralo.

En Mac:

- Pulse **Cmd + Espacio**, escriba **Terminal** y pulse Intro.

Aparecerá una ventana con texto y un cursor intermitente. En ella se introducirán los comandos que se indican a continuación.

💡 Para **pegar** texto en la terminal: en Windows, pulse el botón derecho del ratón; en Mac, **Cmd + V**. Tras pegar cada comando, pulse **Intro**.

Paso 3 · Instalación de Python (parte 1)

1. Acceda a <https://www.python.org/downloads/>
2. Pulse el botón principal para descargar la **versión más reciente** de Python (la 3.14; también valen la 3.11, 3.12 o 3.13).
3. Abra el archivo descargado para iniciar la instalación.

 **En Windows es imprescindible:** en la primera pantalla del instalador, active la casilla **Add Python to PATH** (situada en la parte inferior) **antes** de pulsar *Install Now*. De lo contrario, los comandos no funcionarán.

Paso 3 · Instalación de Python (parte 2: verificación)

Una vez finalizada la instalación, regrese a la terminal para comprobar que se ha realizado correctamente. Escriba lo siguiente y pulse Intro:

En Windows:

```
python --version
```

En Mac:

```
python3 --version
```

El **resultado esperado** es un texto similar a **Python 3.12.4**. Si el número es **3.10 o superior**, la instalación es correcta.

✗ Si aparece *"command not found"* o *"no se reconoce..."*, significa que en Windows no se activó la casilla *Add Python to PATH*: desinstale Python y repita el Paso 3 activándola.

Paso 4 · Creación de la cuenta de GitHub

Para la herramienta que descargará las imágenes y el modelo (Paso 5) se necesita una cuenta de GitHub (es **gratuita**).

1. Acceda a <https://github.com/signup>
2. Introduzca un **correo electrónico**, una **contraseña** y un **nombre de usuario**. Conserve estos datos.
3. Confirme la dirección de correo (recibirá un mensaje con un código).

💡 El material es de **acceso público**: con la cuenta creada es suficiente, no se requiere ningún permiso ni invitación especial.

Paso 5 · Instalación de `gh` (parte 1)

`gh` es la herramienta encargada de descargar las imágenes desde GitHub.

En Windows (en PowerShell, pegue y pulse Intro):

```
winget install --id GitHub.cli
```

En Mac (requiere "Homebrew"; si no dispone de él, ejecute primero el siguiente comando y siga las indicaciones que aparezcan):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Y a continuación:

```
brew install gh
```



Cierre y vuelva a abrir la terminal al finalizar, para que reconozca la nueva herramienta.

Paso 5 · Conexión de **gh** con la cuenta (parte 2)

A continuación se conecta **gh** con la cuenta de GitHub creada en el Paso 4. Escriba:

```
gh auth login
```

Se formularán varias preguntas. Respóndalas mediante las **flechas** e **Intro**:

1. *What account?* → **GitHub.com**
2. *Preferred protocol?* → **HTTPS**
3. *Authenticate Git...?* → **Yes**
4. *How to authenticate?* → **Login with a web browser**

Aparecerá un **código** de 8 caracteres (p. ej. **A1B2-C3D4**). **Cópielo**, pulse Intro, se abrirá el navegador, **péguelo** y pulse **Authorize**.

✓ Al regresar a la terminal debe figurar el mensaje *"Logged in as [su usuario]"*.

Paso 6 · Acceso a la carpeta de la práctica

La carpeta **CP1-carriles** es la que **descargó y descomprimió en el Paso 1**. Es necesario indicar a la terminal que acceda a ella.

Escriba **cd** (que significa *cambiar de carpeta*), un **espacio**, y a continuación **arrastre la carpeta CP1-carriles** desde el explorador de archivos **hasta la ventana de la terminal**: la ruta se completará automáticamente. Pulse Intro.

```
cd (la ruta se completa al soltar la carpeta)
```

💡 Recomendación: arrastrar la carpeta hasta la terminal evita tener que escribir rutas extensas manualmente y reduce el riesgo de error.

Para confirmar el acceso, escriba **ls** (Mac) o **dir** (Windows): deberán aparecer los elementos **notebooks**, **requirements.txt**, etc.

Paso 7 · Creación del entorno e instalación de dependencias

💡 ¿Qué es un "entorno virtual"? Un **conjunto de herramientas independiente**, específico para esta práctica, que evita interferencias con el resto del equipo. Se crea mediante un único comando.

Introduzca los siguientes comandos **de uno en uno** (pulsando Intro tras cada uno):

Creación del entorno (Windows utiliza `python` ; Mac, `python3`):

```
python -m venv .venv
```

Activación del entorno:

```
.venv\Scripts\activate    ← Windows  
source .venv/bin/activate ← Mac
```

✅ Cuando el entorno está activo, aparece `(.venv)` al principio de la línea. Si no aparece, no continúe.

Paso 7 · Instalación de dependencias (continuación)

Con el entorno activado (`(.venv)` visible), instale todos los componentes que requieren los programas:

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

Este proceso tarda **algunos minutos** y muestra abundante texto en pantalla. Es el comportamiento previsto. Espere a que el cursor vuelva a quedar disponible.

✗ Si el proceso finaliza en **color rojo con la palabra `error`**, copie el mensaje y consúltelo en el foro. Si finaliza con **`Successfully installed ...`**, la instalación es correcta.

🍏 **Mac con chip M1/M2/M3:** si se produce un error relacionado con OpenCV, ejecute **`pip install opencv-python-headless`**.

Paso 8 • Descarga de las imágenes y del modelo

Un único comando descarga las 14 imágenes de carretera y el modelo de inteligencia artificial (alojados en GitHub, de ahí la necesidad de `gh`):

```
python scripts/download_assets.py
```

✓ Al finalizar debe indicarse que se han descargado las imágenes y el archivo `.onnx` (el modelo). Si la descarga falla, compruebe su conexión a internet y que `gh` figura como conectado (Paso 5).

Paso 9 · Apertura de Jupyter

💡 **¿Qué es Jupyter?** Un programa que abre los **notebooks** en el navegador. Un **notebook** es un documento que combina explicaciones y fragmentos de programa que pueden ejecutarse por partes, denominadas **celdas**.

Escriba:

```
jupyter notebook
```

Se abrirá **automáticamente** una pestaña en el navegador con un listado de archivos. Acceda a la carpeta **notebooks** y pulse sobre **01_setup.ipynb**.

⚠️ **No cierre la ventana de la terminal** mientras trabaja: es la que mantiene Jupyter en funcionamiento.

Paso 10 · Ejecución de un notebook

Un notebook se compone de **celdas** (recuadros) dispuestas en secuencia. Para ejecutar una celda:

1. Pulse **dentro** de la celda.
2. Pulse **Mayús + Intro** (la tecla de mayúsculas e Intro simultáneamente).
3. La celda se ejecuta y el foco pasa a la siguiente. El resultado aparece debajo.

Repita la operación con **todas las celdas, de arriba abajo y en orden**.

💡 El indicador **[]** a la izquierda de cada celda: vacío = sin ejecutar; **[*]** = en ejecución; **[5]** = ejecutada (el número indica el orden).

Paso 11 · Verificación final

Ejecute **todas** las celdas de `01_setup.ipynb`. La última debe mostrar:

```
✓ Setup OK – listo para 02
```

Si aparece este mensaje, el equipo está correctamente configurado y **la fase de preparación ha concluido**.

Si, por el contrario, se muestra un **✗** o un error en rojo, identifique la celda que ha fallado y consulte el apartado siguiente.

sos Resolución de incidencias

Síntoma	Causa habitual	Solución
<code>command not found</code> / "no se reconoce"	Python o <code>gh</code> mal instalados	Repita el paso; en Windows, active <i>Add to PATH</i>
No aparece <code>(.venv)</code>	El entorno no se activó	Repita el comando de <i>activación</i> del Paso 7
<code>error</code> en rojo durante <code>pip install</code>	Falló una dependencia	Copie el mensaje al foro
La descarga falla	Sin conexión o <code>gh</code> sin conectar	Compruebe internet y repita <code>gh auth login</code> (Paso 5)
Una celda tarda más de 10 min	Existe un problema	Deténgase y consulte

No se espera que el alumno resuelva estas incidencias por su cuenta. Durante la sesión en directo y en el foro se ofrece asistencia.

PARTE 1 · EL DETECTOR CLÁSICO

Notebook 02_clasico_canny_hough · ~15 min

Función de este notebook

Detecta los carriles **sin inteligencia artificial**, únicamente mediante una secuencia de operaciones sobre la imagen. El notebook guía el proceso paso a paso:

```
Imagen → blanco y negro → suavizado → detección de bordes
                                     ↓
Trazado del carril ← unión de líneas ← detección de rectas ← recorte de zona
```

El alumno **ejecuta cada celda y observa** el efecto sobre la imagen en cada etapa. Se recomienda leer los textos explicativos intercalados entre las celdas.

Durante **décadas**, este fue el procedimiento habitual de detección de carriles en los vehículos. Se comprobará que resulta eficaz... hasta cierto punto.

Actividad: experimentación

En algunas celdas figuran **valores numéricos modificables** (por ejemplo, los "umbrales" del detector de bordes). El notebook invita a probar distintos valores y observar su efecto.

- Modifique un valor, ejecute de nuevo esa celda [**Mayús + Intro**] y **observe cómo cambia la imagen**.
- No existe un valor "correcto" único: el objetivo es que el alumno **determine** una combinación que permita distinguir con claridad los carriles y reduzca el ruido.

 **Anote** los valores seleccionados **y su justificación**. Este aspecto se valora en la entrega: se premia la **decisión razonada**, no la coincidencia exacta con un valor determinado.

Aspectos a observar

Finalmente, el notebook aplica el detector a las **14 imágenes**, clasificadas en tres niveles: **easy** (fáciles), **medium** (intermedias) y **hard** (difíciles). Se recomienda prestar atención al **número de aciertos en cada nivel**.

Considere las siguientes cuestiones (se responderán en la entrega; por ahora, basta con observarlas):

1. ¿Se obtienen numerosos aciertos en las imágenes **fáciles** y escasos en las **difíciles**? ¿Era un resultado previsible?
2. Seleccione **una imagen difícil en la que el detector falle** y analícela: ¿qué la dificulta? ¿Una sombra? ¿Una línea discontinua? ¿Una curva pronunciada?

⊘ No elimine la carpeta **outputs** que se genera: el notebook 04 utiliza la información que se guarda en ella.

PARTE 2 · EL DETECTOR CON INTELIGENCIA ARTIFICIAL

Notebook 03_deep_learning · ~10 min

Función de este notebook

Emplea una **red neuronal previamente entrenada** (por el equipo docente) para detectar los carriles en **las mismas 14 imágenes**. El alumno **no entrena ningún modelo** ni necesita conocer su funcionamiento interno: se limita a cargarlo y utilizarlo.

La finalidad es **comparar el comportamiento** con el detector clásico:

- ¿Es más rápido o más lento?
- ¿Acierta en imágenes difíciles en las que el método clásico fallaba?
- ¿Existe **alguna** imagen en la que el método clásico resulte superior? (analízelo con rigor)

El notebook realiza todo el procesamiento. El alumno ejecuta las celdas en orden y observa los resultados.

💡 Un concepto relevante: la velocidad

Una cámara de un vehículo capta aproximadamente **30 imágenes por segundo**. Esto implica que el equipo dispone únicamente de **33 milésimas de segundo** para procesar cada imagen sin acumular retraso.

El notebook **mide el tiempo** que emplea la red neuronal por imagen y lo contrasta con dicho límite.

👉 **Anote** el resultado: ¿logra la inteligencia artificial procesar a tiempo en su equipo, o no alcanza dicho ritmo? Es una de las cuestiones de la entrega, y el resultado suele resultar **llamativo**.

Analice asimismo **tres imágenes concretas**: una en la que ambos métodos acierten, una imagen difícil en la que la inteligencia artificial corrija el fallo del método clásico, y una en la que **la inteligencia artificial también falle**. ¿Qué característica de esa imagen la induce al error?

PARTE 3 · LA COMPARACIÓN

Notebook 04_comparativa · ~10 min

Función de este notebook

Este notebook **no vuelve a procesar imágenes**: recupera los resultados guardados por los notebooks 02 y 03 y **genera las gráficas y la tabla** que se incluirán en la entrega. Al ejecutar sus celdas en orden se crearán, dentro de la carpeta **outputs**, tres imágenes:

- **Un mosaico** que muestra **imagen original | método clásico | inteligencia artificial** en paralelo.
- **Un gráfico de barras** con los aciertos por nivel de dificultad.
- **Un gráfico de tiempos** que compara la velocidad de cada método.

Estas **tres imágenes** son las que deberán incorporarse al documento PDF de entrega. Consérvelas.

Elementos a analizar en las gráficas

- **Gráfico de aciertos:** ¿la ventaja de la inteligencia artificial **aumenta** a medida que las imágenes se vuelven más difíciles, o se mantiene constante?
- **Gráfico de tiempos:** ¿cuántas **veces** más lento es un método respecto al otro?
- **Mosaico:** identifique la fila en la que el método clásico traza un carril **claramente erróneo** y la inteligencia artificial lo resuelve correctamente; y, en su caso, el ejemplo contrario.

Es en este punto donde se aprecia con claridad la diferencia entre ambos métodos. Se recomienda dedicar un momento a cada gráfica.

PARTE 4 · LA ENTREGA

Documento a subir a Moodle

Contenido de la entrega

Un **único archivo PDF** con el nombre `cp1_<apellido>_<nombre>.pdf`, subido a **Moodle** (sección "CP1 — Carriles"), en un plazo de **48 horas** tras la sesión en directo.

El documento deberá contener las siguientes secciones:

1. **Equipo empleado** — una frase: qué equipo se ha utilizado y cuánto tiempo ha requerido la práctica.
2. **Detector clásico** — 5 imágenes con los carriles trazados + los valores seleccionados y su justificación.
3. **Detector con inteligencia artificial** — las mismas 5 imágenes + observaciones (tiempo, aciertos).
4. **Comparación** — las 3 gráficas + un párrafo de conclusión.
5. **Respuestas** a las 5 preguntas del archivo `05_preguntas.md`.

💡 Use la **plantilla** `plantilla-respuestas-cp1.docx` (carpeta `entregable/`): trae las secciones y las preguntas ya montadas. Rellénela, inserte las imágenes y expórtela a PDF (**Archivo** → **Exportar** → **PDF**).

Las 5 preguntas, en síntesis

El archivo `05_preguntas.md` las presenta con mayor detalle. En resumen:

1. **¿En qué situaciones es superior la inteligencia artificial?** — cite 3 imágenes concretas (por su nombre) en las que el método clásico falle, y explique **el motivo**.
2. **¿En qué situaciones es superior el método clásico?** — la cuestión de mayor complejidad. Si no encuentra ninguna, **indíquelo y razone** la causa. La honestidad en el análisis **se valora positivamente**.
3. **Velocidad** — ¿qué método es más rápido? ¿Alcanzan el límite de las 33 milésimas de segundo?
4. **Toma de decisión** — si tuviera que elegir un único método para un vehículo de bajo coste, ¿cuál seleccionaría y por qué?
5. **El caso límite** — proponga una situación de carretera que **induzca al error a ambos métodos** y explique el motivo en cada caso.

No se busca una respuesta "correcta" única, sino un **razonamiento fundamentado** en lo observado.

Criterios de evaluación (16 puntos)

Criterio	Descripción
Ejecución	Ejecución de los notebooks y experimentación con valores propios
Comprensión	Explicación del motivo , no únicamente del resultado
Análisis crítico	Identificación de las limitaciones de cada método
Comunicación	Documento ordenado, con gráficas y texto claro

Aspectos que **elevan la calificación**: datos medidos por el propio alumno, imágenes citadas **por su nombre y honestidad** respecto a lo que no ha funcionado.

Normas de la actividad

- Se **permite el uso de herramientas de inteligencia artificial** (ChatGPT, Claude, Copilot) como apoyo, práctica habitual en el entorno profesional actual. **No obstante**, las conclusiones deben ser **propias**. La reproducción literal de respuestas generadas por dichas herramientas es detectable y penaliza la calificación.
- En caso de haber utilizado una herramienta de inteligencia artificial, **indíquelo** en una línea al final del documento (especificando su finalidad). Declararlo no reduce la calificación; **omitirlo, sí**.
- Existe una carpeta **soluciones/**. Se recomienda **no consultarla** hasta haber realizado la práctica: el objetivo es el razonamiento propio, no la reproducción de la solución.

PARA COMENZAR

Orden de ejecución: 01 → 02 → 03 → 04 → responder 05 .

Ante cualquier dificultad, consulte el apartado "**Resolución de incidencias**" y el **foro del máster**. El acompañamiento docente forma parte de la actividad.

"El método clásico es sumamente rápido. La cuestión es si su precisión resulta suficiente."